

# NODEIMPORT: Imbalanced Node Classification with Node Importance Assessment

Nan Chen  
Johns Hopkins University  
Baltimore, Maryland, USA  
nchen38@jh.edu

Zemin Liu  
Zhejiang University  
Hangzhou, Zhejiang, China  
liu.zemin@zju.edu.cn

Bryan Hooi  
National University of Singapore  
Singapore, Singapore  
bhooi@comp.nus.edu.sg

Bingsheng He  
National University of Singapore  
Singapore, Singapore  
hebs@comp.nus.edu.sg

Jun Hu  
National University of Singapore  
Singapore, Singapore  
jun.hu@nus.edu.sg

Jia Chen  
Grabtaxi Holdings Pte Ltd  
Singapore, Singapore  
jia.chen@grab.com

## Abstract

In real-world applications, node classification on graphs often faces the challenge of class imbalance, where majority classes dominate training, resulting in biased model performance. Traditional Graph Neural Networks (GNNs) often struggle in such scenarios, as they tend to overfit to majority classes while underrepresenting minority classes. Existing solutions, which either prioritize nodes based on class size or synthesize new nodes for minority classes, often fall short of effectively addressing this imbalance issue. This paper introduces a novel approach to class-imbalanced node classification by utilizing a balanced meta-set for importance measurement, where a training node is considered significant if it enhances model performance under an unbiased setting. Our method identifies important nodes that can counteract class imbalance and utilizes them for model training, allowing for fine-grained and dynamic node selection throughout the training process. We theoretically derive a formula to directly assess node importance, reducing computational overhead and providing an intuitive threshold for node selection. Guided by this metric, we develop a novel framework that filters valuable labeled, unlabeled, and synthetic nodes that enhance model performance in an unbiased context. A key advantage of this framework is its separation of the synthetic node generation process from the filtering process, ensuring compatibility with various node generation techniques. Furthermore, we introduce a strategy to construct a high-quality meta-set that closely approximates the overall feature distribution, ensuring robust representation of each class. We evaluate our framework, NODEIMPORT, across multiple benchmark datasets using popular GNN architectures, demonstrating its superiority over state-of-the-art baselines. Our results highlight the flexibility and effectiveness of the framework in mitigating class imbalance, leading to improved node classification outcomes. The source code is available at <https://github.com/NanChanNN/NodeImport>.

## CCS Concepts

• **Computing methodologies** → **Learning latent representations; Neural networks**; • **Mathematics of computing** → **Graph algorithms**.

## Keywords

Graph Neural Networks; Node Classification; Class Imbalanced Learning

## ACM Reference Format:

Nan Chen, Zemin Liu, Bryan Hooi, Bingsheng He, Jun Hu, and Jia Chen. 2025. NODEIMPORT: Imbalanced Node Classification with Node Importance Assessment. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709215>

## 1 Introduction

Graph data is prevalent across many domains, making graph analysis, particularly node classification, a significant research focus [2, 51]. With the rise of deep learning, Graph Neural Networks (GNNs) [15, 20, 46, 52] have become the go-to method for node-level graph analysis, achieving notable success on benchmark datasets. However, GNNs often falter when applied to real-world graphs with imbalanced node class distributions [6, 44, 61, 62, 64]. In these scenarios, majority classes have significantly more labeled nodes for training than minority classes, leading to performance bias that favors majority classes and marginalizes minority classes, resulting in sub-optimal classification outcomes.

To address class-imbalanced node classification, researchers have developed various solutions, which can be broadly categorized into algorithm-level and data-level approaches [27]. Most of these methods can be regarded as identifying and prioritizing important nodes during training to mitigate the imbalance issue. Algorithm-level approaches enhance the importance of minority classes by assigning higher weights to them or expanding their margins [5, 17, 18, 36, 44]. However, these methods typically determine a node's importance based solely on its class size, disregarding its specific features or position within the graph. This oversight of intrinsic differences among nodes of the same class oversimplifies the problem and poses challenges to model training. For instance, outliers within minority classes may introduce noise, and overemphasizing these outliers can skew the model towards misleading patterns. Additionally, certain instances within majority classes, especially those near class



This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '25, August 3–7, 2025, Toronto, ON, Canada  
© 2025 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-1245-6/25/08  
<https://doi.org/10.1145/3690624.3709215>

boundaries, may be crucial in forming a clear decision boundary that benefits both majority and minority classes. Furthermore, static weighting mechanisms are agnostic to the model’s current status and cannot adapt to the model’s training progress. Since neural networks tend to learn patterns of increasing complexity as training progresses [55], a dynamic weighting mechanism that adapts to the model’s learning progress is preferable to a fixed scheme.

On the other hand, data-level approaches balance class sizes by oversampling minority classes through node synthesis [23, 25, 33, 50, 61]. Most methods follow a MixUp-like style [47, 58], sampling pairs of nodes and mixing their features and neighbors to generate new nodes. The generation process serves as an implicit measure of importance, with various heuristics refining each stage of the synthesis pipeline to produce potentially important nodes. However, data-level approaches have limited flexibility, as their implicit importance measures cannot be easily extended to unlabeled nodes, which have significant potential to augment the dataset and address the class imbalance issue [1, 11, 43, 62].

In this work, we propose a different strategy to identify valuable nodes for training to address class-imbalanced node classification, using a balanced meta-set for importance measurement. Our approach is based on a simple yet fundamental assumption [31, 36, 37, 42]: a training node is deemed important if the model, after being trained on it, performs better under an unbiased setting. This strategy offers two main advantages. First, by evaluating the contributions of individual nodes to model performance in a balanced scenario, we can dynamically distinguish important nodes from negligible ones within a class at different training phases. Second, this assumption allows for increased flexibility. For instance, with node oversampling, we can utilize any off-the-shelf node synthesis techniques to generate new nodes and then use the balanced meta-set to identify those that can alleviate class imbalance.

However, straightforwardly applying a meta-set for node filtering in graph-structured data poses unique challenges, leading to two urgent issues. First, assessing node importance with a meta-set requires a bi-level evaluation process: training the model on each node individually and then evaluating the trained model on the meta-set. This process is computationally demanding and time-consuming, which calls for a relaxation of the bi-level evaluation process to achieve higher efficiency. Second, in class-imbalanced node classification scenarios, the balanced meta-set we could construct is generally quite small, making its quality sensitive to random noise. Since the quality of the meta-set is crucial to the overall performance, a carefully designed method is needed to build a high-quality meta-set from the training set.

To tackle these issues, we introduce NODEIMPORT, a versatile framework for class-imbalanced node classification tasks that employs an unbiased meta-set to identify important nodes for training. By expanding the bi-level evaluation process and adopting a series of graph-specific assumptions, we derive a formula to directly calculate the importance of a node in the graph, thereby reducing computational overhead. The derived formula indicates that node importance is influenced not only by static graph characteristics but also by dynamic model prediction behaviors, providing an intuitive interpretation of the formula. Moreover, this formula naturally gives rise to a meaningful threshold for node selection, eliminating the need for manual threshold tuning. Leveraging our importance

formula, we can seamlessly incorporate unlabeled and synthetic nodes into model training, in addition to labeled nodes. Since the computation of node importance is orthogonal to the node generation process, we decouple the node generation process from the filtering process, allowing us to use any existing methods to synthesize nodes or create pseudo-labels for unlabeled nodes, while the filtering process, using the importance formula, then selects valuable nodes able to address class imbalance. Furthermore, to build a high-quality meta-set, we conduct clustering within the embedding space of each class and select representative samples as meta-samples to capture the intrinsic patterns of each class.

We perform a comprehensive empirical evaluation of NODEIMPORT using three popular GNN architectures: GCN [20], GAT [46], and GraphSAGE [15]. This evaluation utilizes multiple benchmark datasets, comprising three citation datasets [56] and two Amazon co-purchase datasets [39], under a long-tailed imbalance setting [9]. The experimental results highlight the effectiveness of the derived importance formula for selecting valuable nodes for training. In addition, it demonstrates the efficiency of the proposed framework in incorporating various node sources to augment the training set and address class imbalance in node-level classification tasks. Overall, the major contributions of this work are summarized as:

- To address class imbalance in node classification, we propose using a balanced meta-set to select nodes valuable to model training. This approach allows a detailed examination of individual nodes’ contributions and provides increased flexibility in the node selection process.
- We theoretically derive a formula to directly assess node importance, thus largely reducing the computational overhead. Furthermore, this formula naturally gives rise to a filtering threshold and has an intuitive interpretation.
- Leveraging the derived formula, we introduce NODEIMPORT, a versatile framework that incorporates valuable labeled, unlabeled, and synthetic nodes for training to counter class distribution imbalance. Additionally, we build a component to extract a high-quality meta-set from labeled nodes.
- Comprehensive experiments on various public benchmark datasets demonstrate the superiority of NODEIMPORT compared to state-of-the-art baselines. Detailed analyses highlight the necessity and effectiveness of its components.

## 2 Related works

**Class imbalance classification.** Existing solutions to the class imbalance problem can be broadly categorized into algorithm-level and data-level approaches [27]. Algorithm-level methods aim to guide the model to focus more on minority classes. This can be achieved by assigning higher weights to minority classes [9, 14, 18, 24, 53] or by expanding the margins for minority classes [3, 17, 30, 36, 45]. However, these methods gauge a training sample’s importance according to its class size, treating all samples from minority classes as equally important without considering their intrinsic characteristics. Although some methods tailored for graph data, such as TAM [44] and ReNode [5], consider a node’s topological

location when assigning weights, their weights do not dynamically adapt to the model’s status during training. Other algorithm-level approaches include imposing additional regularization constraints [40, 54], employing multi-expert training with knowledge distillation [57], or by correcting nodes with ineffective message passing caused by topological disparities. [28].

Data-level techniques balance class sizes through resampling [8, 26, 57, 60] or generative methods [4, 35, 48]. In the graph domain, most works [23, 25, 33, 50, 61] adopt MixUp-like [47, 58] generative methods to augment minority classes, differing in how they sample node pairs, mix node features, and construct edges for synthetic nodes. These generative techniques synthesize potentially important minority nodes based on heuristics adopted in the synthesis pipeline. However, these heuristics implicitly define an importance measure that cannot be directly applied to unlabeled nodes, thus limiting their flexibility. Recently, GraphSR [62] was proposed to identify valuable unlabeled nodes to augment minority classes, but its rules cannot be extended to evaluate synthetic nodes. In contrast, our work employs a balanced meta-set to identify important nodes capable of handling class imbalance for training, positing that a training node is essential if it can improve the model’s performance under an unbiased condition.

For an in-depth discussion of class imbalance learning, please kindly refer to the following surveys [16, 22, 27, 29, 59].

**Meta-set guided data selection.** In the image domain, several works employ a balanced meta-set to learn instance-wise weights [37, 42] or class-wise sampling rates [36] to counter class imbalance. However, these approaches often require bi-level optimization during training, imposing a significant computational burden. On the contrary, we focus on graph-structured data and theoretically derive a formula to simplify the bi-level optimization process, thus saving computing resources. In addition, the derived formula naturally gives a filtering threshold and has a specific interpretation related to the graph domain.

In the active learning domain, meta-sets are used to identify valuable training nodes [19, 21, 31], aiming to train a model with as few samples as possible to speed up the training process. Meta-sets in these works usually bear the same class distribution as the training set and do not need to be balanced. The most relevant work to our derived formula is the RHO-Loss [31], which derives a formula to simplify the computation of sample importance. However, the RHO-Loss still requires an extra model trained on the meta-set and a pre-defined filtering threshold. By contrast, our work customizes the RHO-Loss to the graph domain, deriving a formula with semantic meaning specific to graphs. This formula is more straightforward and does not require storing an additional model trained on the meta-set, thus cutting down memory consumption. Additionally, it naturally provides a filtering threshold for node selection, eliminating the need for manual threshold tuning.

### 3 Preliminary

**Graph representation learning.** A graph  $G$  is represented as  $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, X\}$ , where  $\mathcal{V}$  is the set of  $n$  nodes,  $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$  is the set of edges, and  $X \in \mathbb{R}^{n \times d}$  is the node feature matrix with each row  $x_v \in \mathbb{R}^d$  being the feature vector for node  $v \in \mathcal{V}$ . The graph structure can also be described by an adjacency matrix  $A \in$

$\{0, 1\}^{n \times n}$ , where  $A_{i,j} = 1$  if there is an edge  $\langle i, j \rangle$  and  $A_{i,j} = 0$  otherwise. Each node  $v$  is associated with a one-hot encoded label vector  $y_v \in \{0, 1\}^{1 \times c}$  over  $c$  classes, and  $Y \in \{0, 1\}^{n \times c}$  is the label matrix containing these label vectors. Given a graph encoder  $g(\cdot; \theta)$  with parameters  $\theta$  (e.g., a GNN model), nodes within the graph can be mapped into probability distributions  $H \in \mathbb{R}^{n \times c}$  indicating the likelihood of each node belonging to different classes, formulated as  $H = g(\mathcal{G}; \theta)$ . We denote the predictive distribution of the GNN model on node  $v$  as  $p(y_v|v; \theta)$  for the ease of analysis.

**Class-imbalanced node classification.** Class-imbalanced node classification involves categorizing nodes into their respective classes when the label distribution among classes is uneven, often resulting in a performance bias favoring majority classes [41, 61]. The imbalance ratio  $IR = r_0/r_1$  quantifies the degree of imbalance, where  $r_0$  and  $r_1$  represent the sizes of the largest and smallest classes, respectively [27]. The optimal parameters of the graph encoder  $\theta^*$  are obtained by minimizing the cross-entropy loss on the training set  $\mathcal{D}_{tr} = \{(v, y_v)\}$  defined as:

$$\theta^* = \arg \min_{\theta} \sum_{(v, y_v) \in \mathcal{D}_{tr}} -\log p(y_v|v; \theta). \quad (1)$$

Here, we utilize  $L[y_v|v; \theta] = -\log p(y_v|v; \theta)$  to denote the cross-entropy loss for an individual training point  $(v, y_v) \in \mathcal{D}_{tr}$ .

**Importance assessment using meta-sets.** We introduce an unbiased meta-set  $\mathcal{D}_{meta} = \{(v^{(meta)}, y_v^{(meta)})\}$  with a balanced class distribution for evaluating node importance. For simplicity, this meta-set is written as  $\mathbf{v}^{(meta)}$  and  $\mathbf{y}^{(meta)}$ . The meta-set is usually much smaller than the training set (i.e.,  $|\mathcal{D}_{tr}| \gg |\mathcal{D}_{meta}|$ ), and helps rectify the graph encoder in cases of imbalanced training data. Given a training point  $(v, y_v) \in \mathcal{D}_{tr}$ , we define the following metric  $\eta_v$  to evaluate its importance to model training:

$$\begin{aligned} \eta_v &= \log \frac{p(\mathbf{y}^{(meta)}|\mathbf{v}^{(meta)}; \theta_t; (v, y_v))}{p(\mathbf{y}^{(meta)}|\mathbf{v}^{(meta)}; \theta_t)} \\ &= \log p(\mathbf{y}^{(meta)}|\mathbf{v}^{(meta)}; \theta_t; (v, y_v)) \\ &\quad - \log p(\mathbf{y}^{(meta)}|\mathbf{v}^{(meta)}; \theta_t). \end{aligned} \quad (2)$$

Intuitively, this metric assesses whether the current model’s performance on the balanced meta-set improves after being trained on the given point. When  $\eta_v > 0$ , the node is considered important and used in the next training step. Calculating this metric involves training the current model on each data point individually and determining if the updated model performs better on the meta-set. This process is computationally intensive, which necessitates a more tractable approximation.

### 4 The Proposed Model: NODEIMPORT

In this section, we start by deriving a tractable formula to approximate Eq. 2, which simplifies the bi-level selection process to pinpoint important nodes. Subsequently, we illustrate how this formula can be utilized to identify crucial labeled, unlabeled, and synthetic nodes to augment the training set. Finally, we introduce our meta-set construction method, which selects representative meta-samples. The entire framework architecture is illustrated in Figure 1, while its pseudocode and complexity analysis are provided in Appendix B.

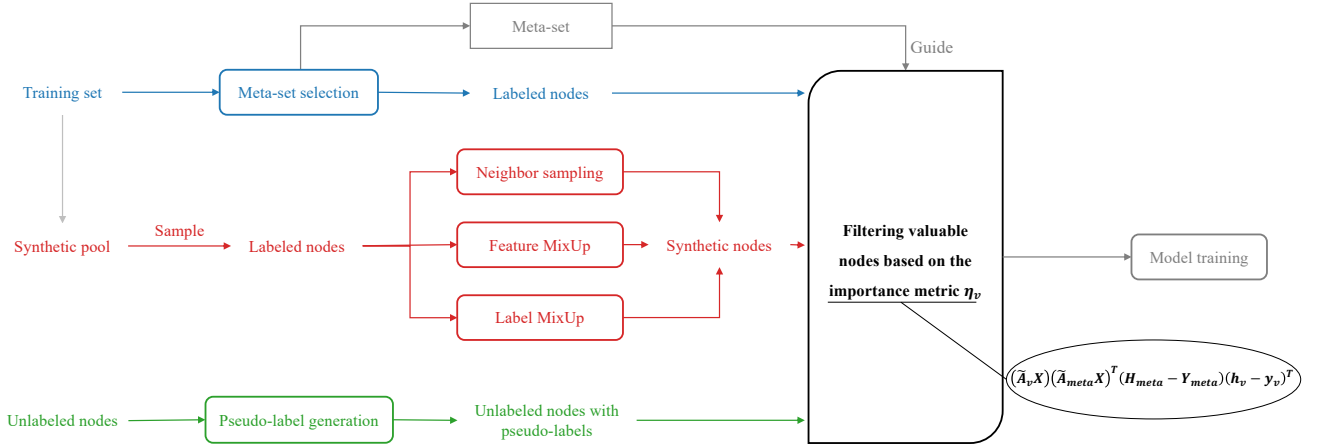


Figure 1: The structure of the proposed framework. The blue, green, and red sections indicate component 1, 2, and 3, respectively.

#### 4.1 Tractable Importance Metric for Node Selection

Following [31], we employ Bayesian probability theory to derive a more tractable form of Eq. 2. By applying Bayes' theorem and assuming conditional independence,  $p(y_v|v, u; \theta_t) = p(y_v|v; \theta_t)$ , we arrive at the following equation:

$$\begin{aligned}
 \eta_v &= \log p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t; (v, y_v)) \\
 &\quad - \log p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t) \\
 &= \log \frac{p(y_v|v; \mathbf{v}^{(meta)}, \mathbf{y}^{(meta)}; \theta_t) \cdot p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}, v; \theta_t)}{p(y_v|v, \mathbf{v}^{(meta)}; \theta_t)} \\
 &\quad - \log p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t) \\
 &= \log \frac{p(y_v|v; \mathbf{v}^{(meta)}, \mathbf{y}^{(meta)}; \theta_t) \cdot p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t)}{p(y_v|v; \theta_t)} \\
 &\quad - \log p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t) \\
 &= \log p(y_v|v; (\mathbf{v}^{(meta)}, \mathbf{y}^{(meta)}); \theta_t) - \log p(y_v|v; \theta_t),
 \end{aligned} \tag{3}$$

where the second equation applies Bayes' theorem to decompose  $p(\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t; (v, y_v))$ , and the third equation simplifies terms using the assumption of conditional independence. Intuitively, the final equation in Eq. 3 can be interpreted as the difference in performance at the training point  $(v, y_v)$  between the current model and the one updated with the meta-set. Let  $\hat{\theta}_t$  denote the parameters of the updated model, we have:

$$\begin{aligned}
 \eta_v &= \log p(y_v|v; \hat{\theta}_t) - \log p(y_v|v; \theta_t) \\
 &= L[y_v|v; \theta_t] - L[y_v|v; \hat{\theta}_t],
 \end{aligned} \tag{4}$$

where, in the second equation, we adopt the definition of cross-entropy loss on point  $(v, y_v)$ . Given Eq. 4, the calculation process simplifies to training the current model on the meta-set and monitoring the change of loss values on training points, which incurs only one extra round of training and is thus more efficient. However, it remains computationally demanding as we need to update the model on the meta-set at each training step.

Suppose that  $\hat{\theta}_t = \theta_t + \Delta$ , where  $\Delta$  represents the parameter updates based on the meta-set, we employ a first-order Taylor series expansion to express  $L[y_v|v; \hat{\theta}_t]$  in terms of  $L[y_v|v; \theta_t]$  and turn Eq. 4 into the following format:

$$\begin{aligned}
 \eta_v &= L[y_v|v; \theta_t] - L[y_v|v; \theta_t + \Delta] \\
 &= L[y_v|v; \theta_t] - (L[y_v|v; \theta_t] + \langle \nabla_{\theta} L[y_v|v; \theta_t], \Delta \rangle_F + o(\Delta^2)) \quad (5) \\
 &\approx -\langle \nabla_{\theta} L[y_v|v; \theta_t], \Delta \rangle_F,
 \end{aligned}$$

where  $\nabla_{\theta} L[y_v|v; \theta_t]$  is the gradient of the individual loss function with regard to weights  $\theta$  at  $\theta_t$ , and  $\langle \cdot, \cdot \rangle_F$  represents the Frobenius inner product. To get a cheaper estimate of parameter updates  $\Delta$ , we assume that a single optimization step is taken for model  $\theta_t$  on the meta-set  $\mathcal{D}_{meta}$ , which gives the following assumption.

**Assumption 4.1.** We update the current model weights  $\theta_t$  on the meta-set  $\mathcal{D}_{meta}$  through one gradient descent step to obtain  $\hat{\theta}_t$ , meaning that  $\hat{\theta}_t = \theta_t - \kappa \cdot \nabla_{\theta} L[\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t]$  where  $\kappa$  is a pre-defined learning rate. Therefore,  $\Delta = -\kappa \cdot \nabla_{\theta} L[\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t]$ .

With the assumption above, the importance of a node is determined by the alignment of gradient descent directions between the training point and the meta-set, which is compatible with findings from previous works on meta-learning [10, 12, 37, 42]:

$$\eta_v \approx \kappa \cdot \langle \nabla_{\theta} L[y_v|v; \theta_t], \nabla_{\theta} L[\mathbf{y}^{(meta)} | \mathbf{v}^{(meta)}; \theta_t] \rangle_F. \tag{6}$$

To obtain a more explicit expression of the cross-entropy loss function  $L[y_v|v; \theta_t]$ , we make the following assumption about the architecture of the GNN encoders used for ease of analysis.

**Assumption 4.2.** We assume that the GNN encoder in use follows the format  $H = \text{SoftMax}(\tilde{A}X\theta)$ , where  $\tilde{A} \in \mathbb{R}^{n \times n}$  is the aggregation matrix,  $\theta \in \mathbb{R}^{d \times c}$  the model parameters, and  $H \in \mathbb{R}^{n \times c}$  the output of the model. Note that the aggregation matrix  $\tilde{A}$  is not necessarily equivalent to the adjacency matrix  $A$ ; it can be any polynomial of  $A$ , such as  $\sum_{k=0}^{\infty} \gamma_k A^k$  in GPR-GNN [7],  $(D^{-\frac{1}{2}} A D^{-\frac{1}{2}})^K$  in SGC [49], or  $\frac{1}{K} \sum_{k=1}^K ((1 - \alpha)(D^{-\frac{1}{2}} A D^{-\frac{1}{2}})^k + \alpha I)$  in SSGC [63].

Given this assumption, the individual loss function  $L[y_v|v; \theta_t]$  can be expressed in terms of the underlying GNN model. Let  $\tilde{A}_v \in$

$\mathbb{R}^{1 \times n}$  be the aggregation matrix associated with node  $v$ , we get:

$$L[y_v|v; \theta_t] = -\text{LOGSOFTMAX}(\tilde{A}_v X \theta) y_v^T, \quad (7)$$

where  $\text{LOGSOFTMAX}$  represents the function that applies element-wise logarithm to the output of softmax. To proceed with the derivation of Eq. 6, it is necessary to compute the derivative of the cross-entropy loss function  $L[\cdot]$  defined in Eq. 7 with respect to the model parameters  $\theta$ . Given  $H = \text{SOFTMAX}(\tilde{A} X \theta)$ , the derivative of the cross-entropy loss with regard to model parameters  $\theta$  is:

$$\nabla_{\theta} L = (\tilde{A} X)^T (H - Y). \quad (8)$$

The detailed derivation of Eq. 8 is provided in Appendix A.1. In essence, we work in differential form to compute the derivative. Substituting Eq. 8 into Eq. 6 gives our final importance metric:

$$\begin{aligned} \eta_v &\approx \kappa \cdot \langle \nabla_{\theta} L[y_v|v; \theta_t], \nabla_{\theta} L[y^{(meta)}|v^{(meta)}; \theta_t] \rangle_F \\ &= \kappa \cdot \text{tr}((h_v - y_v)^T (\tilde{A}_v X) (\tilde{A}_{meta} X)^T (H_{meta} - Y_{meta})) \\ &= \kappa \cdot (\tilde{A}_v X) (\tilde{A}_{meta} X)^T (H_{meta} - Y_{meta}) (h_v - y_v)^T, \end{aligned} \quad (9)$$

where  $\tilde{A}_{meta} \in \mathbb{R}^{m \times n}$ ,  $H_{meta} \in \mathbb{R}^{m \times c}$ , and  $Y_{meta} \in [0, 1]^{m \times c}$  represent the aggregation matrix, model prediction, and label matrix associated with nodes in the meta-set, respectively.

**Understanding the importance metric** We offer an explanation of the functioning of our derived importance metric in Eq. 9 by breaking it down into two constituent components:

- **Context similarity**  $(\tilde{A}_v X) (\tilde{A}_{meta} X)^T$ : This component measures the similarity between the node sample  $(v, y_v)$  and the meta-set  $\mathcal{D}_{meta}$  by considering both the input features of the nodes and the structure of their local subgraphs. It captures the local context similarity within the graph, reflecting static graph characteristics and being agnostic to the underlying GNN model.
- **Prediction behavior similarity**  $(H_{meta} - Y_{meta}) (h_v - y_v)^T$ : This component evaluates the similarity in the prediction behavior of the model between the node sample  $(v, y_v)$  and the meta-set  $\mathcal{D}_{meta}$ . It is relevant to the underlying GNN model and captures the model's dynamics during training.

By examining the importance metric as a whole, we observe that training samples from under-represented classes (i.e., minority classes) are more favored by the metric compared to those from over-represented classes (i.e., majority classes). This is partly because our meta-set is balanced, ensuring that each class has an equal voting opportunity. Additionally, the term  $(H_{meta} - Y_{meta})$  specifically assesses the current model performance on each class and places more emphasis on classes with poor performance. Furthermore, the context similarity and prediction behavior similarity between a node and the meta-samples of its class are also crucial, aiding in the removal of outliers in the minority classes. This explains why our metric is capable of effectively addressing class-imbalanced issues.

**Instantiating the importance metric.** In practice, we employ existing polynomials of the adjacency matrix  $A$  (such as those used in APPNP [13] or SSGC [63]) to calculate the aggregation matrix  $\tilde{A}$  and utilize the calculated matrix to compute the context similarity. It is important to note that the context similarity is computed once at the beginning of the program and is used throughout the entire training process. The prediction behavior similarity is computed using the output of the current model, which requires no additional

computation. In general, applying our formula to evaluate the importance of nodes incurs minimal extra computational overhead.

## 4.2 Framework

Based on the importance metric derived in Eq.9, our primary approach is to apply this metric to labeled nodes. This allows us to filter out valuable nodes for model training, which can effectively counter class imbalance. Additionally, this metric can be extended to include unlabeled and synthetic nodes, which has been proven beneficial for augmenting the training set and addressing class imbalance issues [50, 61, 62]. To utilize this metric, we need to generate pseudo-labels for unlabeled nodes. As for synthetic nodes, we synthesize their input features, edge connections, and labels. A key advantage of our metric lies in its independence from specific methods for generating pseudo-labels and synthesizing nodes, ensuring compatibility with any existing techniques in the literature. In this work, we demonstrate that even the simplest methods yield superior performance when equipped with our metric. In the subsequent sections, we describe how we incorporate valuable labeled, unlabeled, and synthetic nodes into the training set, each of which forms a distinct component.

*Component 1.* The first component we construct is to identify labeled nodes that are worth training in a class-imbalanced setting using our importance metric. Let  $\mathcal{D}_l = \{(v, y_v)\}$  denote the set of labeled nodes. The set of valuable labeled nodes  $\tilde{\mathcal{D}}_l$  is defined as:

$$\tilde{\mathcal{D}}_l = \{(v, y_v) \in \mathcal{D}_l \mid \eta_v > 0\}. \quad (10)$$

*Component 2.* As the labels for the unlabeled nodes are unavailable during training, we generate pseudo-labels for them before applying the importance metric. In this study, we choose the class with the highest prediction probability from the model as the pseudo-label. For each unlabeled node  $v \in \mathcal{D}_{ul}$ , its pseudo-label  $\hat{y}_v$  is:

$$\hat{y}_v = \arg \max_k h_v[k], \quad (11)$$

where  $h_v = H[v, :] \in \mathbb{R}^{1 \times c}$  is the prediction probability distribution of the current GNN model on node  $v$ . With the pseudo-label  $\hat{y}_v$ , we replace  $y_v$  in Eq.9 with it to compute the importance  $\eta_v$  for the unlabeled node. The filtered set of unlabeled nodes  $\tilde{\mathcal{D}}_{ul}$  is then:

$$\tilde{\mathcal{D}}_{ul} = \{(v, \hat{y}_v) \mid v \in \mathcal{D}_{ul} \text{ and } \eta_v > 0\}. \quad (12)$$

*Component 3.* In this study, we adopt a MixUp-like methodology to generate synthetic nodes, augmenting the training dataset to mitigate class imbalance. Firstly, we draw a set of node pairs  $\mathcal{S} = \{((v_s, y_s), (v_t, y_t))\}$  from the training set. Specifically, nodes  $v_s$  are sampled until each class reaches the maximum sample count among all classes in the training set, while nodes  $v_t$  are sampled based on the probability  $p_t(u)$  defined as follows:

$$p_t(u) = \frac{\log(|\mathcal{V}_k| + 1)}{(|\mathcal{V}_k| + 1) \sum_{j=1}^c \log(|\mathcal{V}_j| + 1)}. \quad (13)$$

where  $k$  is the label of node  $u$  and  $|\mathcal{V}_k|$  stands for the subset of nodes with label  $k$ . This increases the likelihood of selecting nodes from minority classes for synthesis, thereby enriching their patterns within the training set. For each node pair  $((v_s, y_s), (v_t, y_t)) \in \mathcal{S}$ , we generate a new node  $v_{syn}$  by employing MixUp [58] from three

aspects, namely feature, neighborhood, and label mixing:

$$\begin{cases} x_{syn} = \lambda \cdot x_s + (1 - \lambda) \cdot x_t, \\ A'_{syn} = \lambda \cdot A_s + (1 - \lambda) \cdot A_t, \\ y_{syn} = \lambda \cdot y_s + (1 - \lambda) \cdot y_t. \end{cases} \quad (14)$$

where  $\lambda \sim \text{Beta}(\alpha, \alpha)$  represents the synthetic scale drawn from the Beta distribution with the parameter  $\alpha$ , and  $A'_{syn} \in \mathbb{R}^{1 \times n}$  denotes the combined adjacency matrix for nodes  $v_s$  and  $v_t$ . Since  $A'_{syn}$  consists of real numbers, it must be converted into a discrete version. This involves sampling a specific number of neighbors for node  $v_{syn}$  based on  $A'_{syn}$ , resulting in a discrete form  $A_{syn}$ :

$$A_{syn} = \text{Sample}(A'_{syn}), \quad (15)$$

where the probability of an item  $A_{syn}[k]$  being one is proportional to the corresponding value in  $A'_{syn}[k]$ . To maintain consistent degree statistics as per common practice in prior research [23, 33], the number of sampled neighbors is drawn from a separate degree distribution of the entire graph. In this way, we produce a set of new nodes, i.e.,  $\mathcal{D}_{syn} = \{(v_{syn}, y_{syn})\}$ , based on the sampled node pairs  $\mathcal{S}$ . Combining Eq. 9 and Eq. 14, we calculate the importance of a synthetic node  $\eta_{v_{syn}}$  to determine whether to use it for training. Finally, we build a set of valuable synthetic nodes:

$$\tilde{\mathcal{D}}_{syn} = \{(v_{syn}, y_{syn}) \in \mathcal{D}_{syn} \mid \eta_{v_{syn}} > 0\}. \quad (16)$$

**Training objective.** The overall training objective  $\mathcal{L}$  is defined as the sum of the cross entropy loss on high-quality labeled nodes  $\mathcal{L}_l$ , unlabeled nodes  $\mathcal{L}_{ul}$ , and synthetic nodes  $\mathcal{L}_{syn}$ :

$$\mathcal{L} = \mathcal{L}_l + \beta \cdot \mathcal{L}_{ul} + \gamma \cdot \mathcal{L}_{syn}, \quad (17)$$

where  $\beta$  and  $\gamma$  are scaling factors for the losses of the unlabeled and synthetic nodes, respectively.

### 4.3 Meta-set construction

The effectiveness of our framework is heavily dependent on the quality of the meta-set due to its small size. Our goal is to select a fixed number of nodes for each class  $k$  that can accurately reflect the distribution of the initial training pool  $\mathcal{V}_k$ . Let  $\mathcal{M}_k \subseteq \mathcal{V}_k$  denote the set of meta-samples for class  $k$ . We aim to find the optimal set  $\tilde{\mathcal{M}}_k$  by minimizing the following objective:

$$\tilde{\mathcal{M}}_k = \arg \min_{\mathcal{M}_k} \frac{1}{|\mathcal{V}_k|} \sum_{v \in \mathcal{V}_k} D(v, \mathcal{M}_k), \quad (18)$$

where  $D(\cdot, \cdot)$  represents the distance between a node  $v$  and the meta-set  $\mathcal{M}_k$ . To calculate this distance, we exploit the context embedding for the nodes,  $F = \tilde{A}X$ , which captures not only the nodes' features but also their local structures. Let  $f_v$  be the context embedding vector of a node  $v$ , and we define the distance  $D(v, \mathcal{M}_k)$  between the node and the meta-set  $\mathcal{M}_k$  as:

$$D(v, \mathcal{M}_k) = \min_{u \in \mathcal{M}_k} \text{dist}(f_v, f_u), \quad (19)$$

where  $\text{dist}(\cdot, \cdot)$  denotes the distance metric between two vectors. In this work, this metric can be the Euclidean distance or the Manhattan distance. Intuitively, the distance between a node and the meta-set is formulated as the distance to its closest meta-sample in the meta-set. Given the impracticality of finding the optimal solution through exhaustive search, we employ a heuristic search method. In particular, we utilize the Partitioning Around Medoids

**Table 1: Statistics of datasets adopted in this work**

| Dataset          | Nodes  | Edges   | Features | Classes |
|------------------|--------|---------|----------|---------|
| Cora             | 2,708  | 10,556  | 1,433    | 7       |
| CiteSeer         | 3,327  | 9,104   | 3,703    | 6       |
| PubMed           | 19,717 | 88,648  | 500      | 3       |
| Amazon-Photo     | 7,650  | 119,081 | 745      | 8       |
| Amazon-Computers | 13,752 | 245,861 | 767      | 10      |

(PAM) algorithm [32] to identify meta-samples for each class. The meta-set utilized is then the union of meta-samples from each class:

$$\mathcal{D}_{meta} = \{(v, y_v) \mid v \in \bigcup_{k=1}^c \tilde{\mathcal{M}}_k\}. \quad (20)$$

In the end, we exclude the meta-set  $\mathcal{D}_{meta}$  from the training set  $\mathcal{D}_{tr}$  to form the set of labeled nodes  $\mathcal{D}_l$ .

## 5 Experiments

### 5.1 Experimental Setup

**Datasets.** We adopt five benchmark datasets, which consist of three citation networks (Cora, CiteSeer, and PubMed) [56] and two Amazon co-purchase networks (Amazon-Photo and Amazon-Computers) [39]. Dataset statistics are summarized in Table 1. For all datasets, we set the imbalance ratio  $IR$  to 50 and construct a long-tailed training set following [9]. For the citation datasets, we use the public data splits from [34] and iteratively remove nodes from the training set to create a long-tailed distribution. For the co-purchase networks, we set the training ratio to 10% and sample nodes for each class to follow a long-tailed distribution. The remaining data is split into a 1:8 ratio for validation and testing. Unlike the citation datasets, the co-purchase datasets are naturally imbalanced, meaning that their validation and test sets are also highly imbalanced.

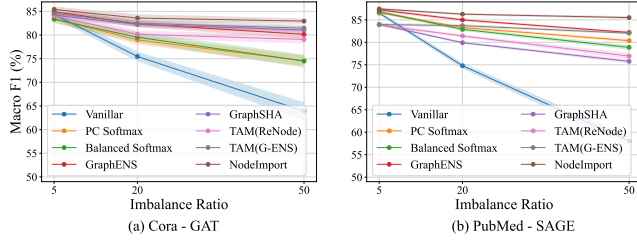
**Baselines.** We compare our model against eleven established baselines, which are categorized into three groups: (1) *Traditional methods*, which include the vanilla approach using conventional cross-entropy loss and GRAND [11], which utilizes unlabeled nodes to enhance model training; (2) *Algorithm-level methods*, consisting of Re-weight [18], Balanced Softmax [36], PC Softmax [17], ReNode [5], TAM [44], ReVar [54], and BAT [28]; (3) *Data-level methods*, including GraphENS [33] and GraphSHA [23]. To ensure a comprehensive evaluation, we integrate TAM with Balanced Softmax, ReNode, and GraphENS.

**Evaluation protocol.** In this study, we deploy both our proposed model and the selected baselines across three representative GNN architectures: GCN [20], GAT [46], and GraphSAGE [15]. Nodes that are not part of the training set are treated as unlabeled in our setting. For the evaluation of model performance, we employ three metrics: Accuracy (Acc.), balanced Accuracy (BAcc.), and the Macro F1 score (Macro F1). All these metrics are scaled between 0 and 1, where higher values represent better model performance. We conduct ten independent runs for each method and report the average score and the standard error for a robust statistical analysis.





GRAND [11] to the vanilla approach, we observe that leveraging unlabeled nodes can boost performance in some cases, highlighting their potential to mitigate class imbalance. However, in most instances, the vanilla approach outperforms GRAND, suggesting that careful handling of unlabeled nodes is crucial under class imbalance conditions to avoid negative impacts on model performance.



**Figure 2: Model performance under varying imbalance ratios in terms of Macro F1 score.**

**Performance under different imbalance ratios.** Figure 2 illustrates the behavior of various methods under different imbalance ratios ( $IR \in \{5, 20, 50\}$ ). As the imbalance ratio decreases, the performance gap in Macro F1 scores among different methods narrows. Conversely, as the imbalance ratio increases, our model’s superiority becomes more pronounced, demonstrating its effectiveness in handling highly imbalanced problems. Even with a small imbalance ratio, our model delivers slight performance improvements, likely due to the integration of valuable unlabeled and synthetic nodes, which helps regularize model training.

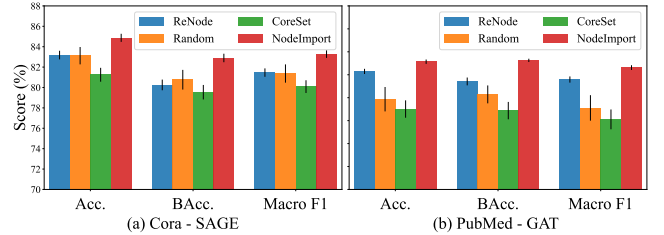
### 5.3 Model Analysis

**Ablation study.** We evaluate the contributions of the three main components introduced in Section 4.2. Table 3 presents the results of an ablation study that assesses the individual and combined impacts of each component. When component 1 is excluded, we train with all labeled nodes without filtering. The results confirm the importance of all three components in enhancing the model’s capacity to address class imbalance in graphs. Each component individually improves performance, with component 3 (synthetic nodes) generally providing the most significant enhancement. This improvement is likely due to the introduction of diverse patterns into minority classes during node synthesis, which helps balance the class distribution. Furthermore, the combination of components 2 and 3 nearly matches the performance of the full model. Adding component 1 to this combination offers a modest incremental benefit, usually within a one-percent range.

**Analysis of the meta-set construction method.** We evaluate the effectiveness of our meta-set construction method proposed in Section 4.3 by comparing it with three alternatives: Random sampling, ReNode [5] (which uses Totoro metrics to select top-ranked nodes), and CoreSet [38] (which iteratively selects nodes farthest from current selections). Results in Figure 3 show that our proposed method consistently outperforms the other techniques across all metrics. Notably, Random sampling and ReNode exhibit similar

**Table 3: Ablation study of model components. ✓ indicates the inclusion of a component, whereas ✗ denotes its exclusion.**

|                      | C1 | C2 | C3 | Acc.                    | bAcc.                   | Macro F1                |
|----------------------|----|----|----|-------------------------|-------------------------|-------------------------|
| Cora + GCN           | ✗  | ✗  | ✗  | 81.55 $\pm 0.51$        | 75.38 $\pm 0.67$        | 77.98 $\pm 0.65$        |
|                      | ✓  | ✗  | ✗  | 83.70 $\pm 0.61$        | 79.92 $\pm 0.80$        | 81.12 $\pm 0.70$        |
|                      | ✗  | ✓  | ✗  | 84.12 $\pm 0.28$        | 81.13 $\pm 0.13$        | 82.01 $\pm 0.27$        |
|                      | ✗  | ✗  | ✓  | 83.72 $\pm 0.56$        | 80.39 $\pm 0.73$        | 81.58 $\pm 0.60$        |
|                      | ✗  | ✓  | ✓  | <b>85.33</b> $\pm 0.33$ | <b>83.85</b> $\pm 0.53$ | <b>83.50</b> $\pm 0.46$ |
|                      | ✓  | ✗  | ✓  | 83.82 $\pm 0.60$        | 81.36 $\pm 0.71$        | 82.05 $\pm 0.60$        |
|                      | ✓  | ✓  | ✗  | 84.53 $\pm 0.41$        | 82.56 $\pm 0.43$        | 82.48 $\pm 0.44$        |
|                      | ✓  | ✓  | ✓  | <b>85.11</b> $\pm 0.42$ | <b>83.71</b> $\pm 0.49$ | <b>83.24</b> $\pm 0.51$ |
|                      |    |    |    |                         |                         |                         |
| CiteSeer + GraphSAGE | ✗  | ✗  | ✗  | 56.07 $\pm 1.65$        | 48.20 $\pm 1.07$        | 45.70 $\pm 1.46$        |
|                      | ✓  | ✗  | ✗  | 62.71 $\pm 1.18$        | 55.67 $\pm 0.80$        | 53.95 $\pm 1.02$        |
|                      | ✗  | ✓  | ✗  | 66.65 $\pm 1.12$        | 59.99 $\pm 0.99$        | 57.91 $\pm 1.05$        |
|                      | ✗  | ✗  | ✓  | 64.01 $\pm 1.35$        | 56.44 $\pm 0.83$        | 54.68 $\pm 1.06$        |
|                      | ✗  | ✓  | ✓  | <b>70.45</b> $\pm 1.04$ | 63.42 $\pm 0.95$        | 61.91 $\pm 1.17$        |
|                      | ✓  | ✗  | ✓  | 65.30 $\pm 1.13$        | 58.14 $\pm 0.86$        | 56.56 $\pm 1.04$        |
|                      | ✓  | ✓  | ✗  | 68.99 $\pm 0.87$        | 62.31 $\pm 0.54$        | 60.98 $\pm 0.69$        |
|                      | ✓  | ✓  | ✓  | <b>71.12</b> $\pm 0.84$ | <b>64.22</b> $\pm 0.62$ | <b>62.89</b> $\pm 0.78$ |
| photo + GraphSAGE    | ✗  | ✗  | ✗  | 90.53 $\pm 0.30$        | 84.87 $\pm 0.49$        | 86.95 $\pm 0.46$        |
|                      | ✓  | ✗  | ✗  | 90.49 $\pm 0.12$        | 84.59 $\pm 0.41$        | 86.04 $\pm 0.43$        |
|                      | ✗  | ✓  | ✗  | 91.16 $\pm 0.30$        | 88.42 $\pm 0.39$        | 89.58 $\pm 0.32$        |
|                      | ✗  | ✗  | ✓  | 90.92 $\pm 0.11$        | 88.01 $\pm 0.63$        | 88.43 $\pm 0.45$        |
|                      | ✗  | ✓  | ✓  | 93.78 $\pm 0.04$        | 92.56 $\pm 0.18$        | 92.49 $\pm 0.08$        |
|                      | ✓  | ✗  | ✓  | 91.11 $\pm 0.09$        | 88.64 $\pm 0.43$        | 88.99 $\pm 0.18$        |
|                      | ✓  | ✓  | ✗  | 91.36 $\pm 0.42$        | 87.93 $\pm 0.95$        | 88.92 $\pm 1.12$        |
|                      | ✓  | ✓  | ✓  | <b>93.94</b> $\pm 0.06$ | <b>93.02</b> $\pm 0.15$ | <b>92.71</b> $\pm 0.10$ |



**Figure 3: Comparison of meta-set construction methods.**

performance levels, while CoreSet performs the worst. The inferior performance of CoreSet is likely due to the inclusion of many outliers in the meta-set, which negatively impacts the performance.

**Case study on per-class F1 scores.** Table 4 presents the per-class F1 scores on the Cora dataset using the GraphSAGE architecture. Baseline methods designed to mitigate class imbalance demonstrate improvements in F1 scores for both minority and majority classes, indicating that addressing class imbalance can refine decision boundaries and benefit all classes. Our model not only achieves the most significant improvements across most classes but also substantially reduces the performance gap between minority and majority classes. Notably, the improvement from our method is particularly pronounced in smaller classes, such as  $C_0$ ,  $C_1$ , and



**Table 4: F1 scores for each class on the Cora dataset using the GraphSAGE backbone, under an imbalance ratio of 50. The table header presents each class, labeled from  $C_0$  to  $C_6$ , with its corresponding distribution located below each class label.**

| Class            | $C_0$        | $C_1$        | $C_2$        | $C_3$        | $C_4$        | $C_5$        | $C_6$        |
|------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Distribution     | 6.80%        | 1.80%        | 25.24%       | 48.54%       | 13.18%       | 3.47%        | 0.97%        |
| Vanilla          | 67.14        | 69.64        | 92.21        | 81.36        | 83.47        | 78.95        | 53.58        |
| GRAND            | 64.57        | 52.59        | 93.10        | 73.54        | 84.34        | 72.94        | 14.75        |
| ReWeight         | 71.68        | 78.88        | 93.43        | 84.13        | 84.45        | 80.49        | 65.88        |
| PC Softmax       | 72.63        | 80.58        | 93.51        | 86.53        | 84.54        | 79.77        | 65.03        |
| Balanced Softmax | 72.18        | 80.84        | 93.12        | 85.41        | 84.36        | 80.83        | 68.38        |
| TAM(BS)          | 72.69        | 80.34        | 92.62        | 85.05        | 84.78        | 81.52        | 70.44        |
| ReNode           | 71.96        | 77.94        | 93.34        | 83.95        | 84.52        | 80.50        | 67.03        |
| TAM(ReNode)      | 73.62        | 78.35        | 92.75        | 83.68        | 85.01        | 81.40        | 70.80        |
| GraphENS         | 72.00        | 81.14        | 93.38        | 85.37        | 83.98        | 79.07        | 68.18        |
| TAM(G-ENS)       | 72.14        | 82.33        | <b>93.54</b> | 85.45        | 84.32        | 81.60        | 70.07        |
| GraphSHA         | 71.28        | 80.09        | 93.19        | 85.43        | 84.73        | 79.91        | 66.00        |
| NODEIMPORT       | <b>75.99</b> | <b>83.84</b> | 93.11        | <b>87.12</b> | <b>86.28</b> | <b>82.48</b> | <b>74.09</b> |

$C_6$ . Compared to the Vanilla method, the GRAND method shows a sharp decrease in the F1 score for the smallest class,  $C_6$ . This suggests that directly using unlabeled nodes for model training without considering class imbalance can lead to the misclassification of unlabeled nodes from extremely small classes like  $C_6$ , further exacerbating the class imbalance issue.

## 6 Conclusions

In this work, we introduce NODEIMPORT, a framework for class-imbalanced node classification. Our method dynamically evaluates node importance and incorporates key nodes into the training process to address class imbalance. We develop an efficient formula for assessing node importance and a framework for selecting high-quality labeled, unlabeled, and synthetic nodes. Additionally, we extract representative meta-samples from the training set. Experiments on benchmark datasets demonstrate that NODEIMPORT outperforms state-of-the-art methods, effectively mitigating class imbalance in node classification tasks.

## Acknowledgments

This research is supported by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative, the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG2-TC-2021-002).

## References

- [1] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A Raffel. 2019. Mixmatch: A holistic approach to semi-supervised learning. *Advances in neural information processing systems* 32 (2019).
- [2] Hongyun Cai, Vincent W Zheng, and Kevin Chen-Chuan Chang. 2018. A comprehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering* 30, 9 (2018), 1616–1637.
- [3] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arachiga, and Tengyu Ma. 2019. Learning imbalanced datasets with label-distribution-aware margin loss. *Advances in neural information processing systems* 32 (2019).

- [4] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research* 16 (2002), 321–357.
- [5] Deli Chen, Yankai Lin, Guangxiang Zhao, Xuancheng Ren, Peng Li, Jie Zhou, and Xu Sun. 2021. Topology-imbalance learning for semi-supervised node classification. *Advances in Neural Information Processing Systems* 34 (2021), 29885–29897.
- [6] Nan Chen, Zemin Liu, Bryan Hooi, Bingsheng He, Rizal Fathony, Jun Hu, and Jia Chen. 2024. Consistency Training with Learnable Data Augmentation for Graph Anomaly Detection with Limited Supervision. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=eMKXvhhQ9>
- [7] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2020. Adaptive universal generalized pagerank graph neural network. *arXiv preprint arXiv:2006.07988* (2020).
- [8] Limeng Cui, Xianfeng Tang, Sumeet Katariya, Nikhil Rao, Pallav Agrawal, Karthik Subbian, and Dongwon Lee. 2022. ALLIE: Active learning on large-scale imbalanced graphs. In *Proceedings of the ACM Web Conference 2022*. 690–698.
- [9] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. 2019. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9268–9277.
- [10] Amir Erfan Eshratifar, David Eigen, and Massoud Pedram. 2018. Gradient agreement as an optimization objective for meta-learning. *arXiv preprint arXiv:1810.08178* (2018).
- [11] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph random neural networks for semi-supervised learning on graphs. *Advances in neural information processing systems* 33 (2020), 22092–22103.
- [12] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*. PMLR, 1126–1135.
- [13] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997* (2018).
- [14] Xinyu Guo, Kai Wu, Xiaoyu Zhang, and Jing Liu. 2024. Automated Loss function Search for Class-imbalanced Node Classification. In *Forty-first International Conference on Machine Learning, ICMML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net. <https://openreview.net/forum?id=O1hmwi51pp>
- [15] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [16] Haibo He and Edwardo A Garcia. 2009. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering* 21, 9 (2009), 1263–1284.
- [17] Youngkyu Hong, Seungju Han, Kwanghee Choi, Seokjun Seo, Beomsu Kim, and Buru Chang. 2021. Disentangling label distribution for long-tailed visual recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 6626–6636.
- [18] Nathalie Japkowicz and Shaju Stephen. 2002. The class imbalance problem: A systematic study. *Intelligent data analysis* 6, 5 (2002), 429–449.
- [19] Krishnateja Killamsetty, Durga Sivasubramanian, Ganesh Ramakrishnan, and Rishabh Iyer. 2021. Glist: Generalization based data subset selection for efficient and robust learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 8110–8118.
- [20] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [21] Andreas Kirsch, Tom Rainforth, and Yarin Gal. 2021. Test distribution-aware active learning: A principled approach against distribution shift and outliers. *arXiv preprint arXiv:2106.11719* (2021).
- [22] Bartosz Krawczyk. 2016. Learning from imbalanced data: open challenges and future directions. *Progress in artificial intelligence* 5, 4 (2016), 221–232.
- [23] Wen-Zhi Li, Chang-Dong Wang, Hui Xiong, and Jiansheng He. 2023. GraphSHA: Synthesizing Harder Samples for Class-Imbalanced Node Classification. *SIGKDD* (2023).
- [24] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*. 2980–2988.
- [25] Jie Liu, Mengting He, Guangtao Wang, Nguyen Quoc Viet Hung, Xuequn Shang, and Hongzhi Yin. 2023. Imbalanced node classification beyond homophilic assumption. *arXiv preprint arXiv:2304.14635* (2023).
- [26] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2021. Pick and choose: a GNN-based imbalanced learning approach for fraud detection. In *Proceedings of the web conference 2021*. 3168–3177.
- [27] Zemin Liu, Yuan Li, Nan Chen, Qian Wang, Bryan Hooi, and Bingsheng He. 2023. A survey of imbalanced learning on graphs: Problems, techniques, and future directions. *arXiv preprint arXiv:2308.13821* (2023).
- [28] Zhining Liu, Ruizhong Qiu, Zhichen Zeng, Hyunsik Yoo, David Zhou, Zhe Xu, Yada Zhu, Kommy Weldemariam, Jingrui He, and Hanghang Tong. 2024. Class-Imbalanced Graph Learning without Class Rebalancing. In *Forty-first International Conference on Machine Learning, ICMML 2024, Vienna, Austria, July 21–27, 2024*.

- OpenReview.net. <https://openreview.net/forum?id=pPnkpVBeZn>
- [29] Yihong Ma, Yijun Tian, Nuno Moniz, and Nitesh V Chawla. 2023. Class-imbalanced learning on graphs: A survey. *arXiv preprint arXiv:2304.04300* (2023).
- [30] Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. 2021. Long-tail learning via logit adjustment. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=37nvqkCo5>
- [31] Sören Mindermann, Jan M Brauner, Muhammed T Razzak, Mrinank Sharma, Andreas Kirsch, Winnie Xu, Benedikt Hölting, Aidan N Gomez, Adrien Morisot, Sebastian Farquhar, et al. 2022. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning*. PMLR, 15630–15649.
- [32] Hae-Sang Park and Chi-Hyuck Jun. 2009. A simple and fast algorithm for K-medoids clustering. *Expert systems with applications* 36, 2 (2009), 3336–3341.
- [33] Joonhyung Park, Jaeyun Song, and Eunho Yang. 2022. GraphENS: Neighbor-Aware Ego Network Synthesis for Class-Imbalanced Node Classification. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=MXEL7i-iru>
- [34] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287* (2020).
- [35] Liang Qu, Huaisheng Zhu, Ruiqi Zheng, Yuhui Shi, and Hongzhi Yin. 2021. Imagin: Imbalanced network embedding via generative adversarial graph networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1390–1398.
- [36] Jiawei Ren, Cunjun Yu, Xiao Ma, Haiyu Zhao, Shuai Yi, et al. 2020. Balanced meta-softmax for long-tailed visual recognition. *Advances in neural information processing systems* 33 (2020), 4175–4186.
- [37] Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. 2018. Learning to reweight examples for robust deep learning. In *International conference on machine learning*. PMLR, 4334–4343.
- [38] Ozan Sener and Silvio Savarese. 2017. Active learning for convolutional neural networks: A core-set approach. *arXiv preprint arXiv:1708.00489* (2017).
- [39] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [40] Min Shi, Yufei Tang, Xingquan Zhu, David Wilson, and Jianxun Liu. 2020. Multi-class imbalanced graph convolutional network learning. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*.
- [41] Min Shi, Yufei Tang, Xingquan Zhu, David Wilson, and Jianxun Liu. 2020. Multi-class imbalanced graph convolutional network learning. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*.
- [42] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. 2019. Meta-weight-net: Learning an explicit mapping for sample weighting. *Advances in neural information processing systems* 32 (2019).
- [43] Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. 2020. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *Advances in neural information processing systems* 33 (2020), 596–608.
- [44] Jaeyun Song, Joonhyung Park, and Eunho Yang. 2022. TAM: topology-aware margin loss for class-imbalanced node classification. In *International Conference on Machine Learning*. PMLR, 20369–20383.
- [45] Jingru Tan, Changbao Wang, Buyu Li, Quanquan Li, Wanli Ouyang, Changqing Yin, and Junjie Yan. 2020. Equalization loss for long-tailed object recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11662–11671.
- [46] Petar Velicković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [47] Vikas Verma, Alex Lamb, Christopher Beckham, Amir Najafi, Ioannis Mitliagkas, David Lopez-Paz, and Yoshua Bengio. 2019. Manifold mixup: Better representations by interpolating hidden states. In *International conference on machine learning*. 6438–6447.
- [48] Jianfeng Wang, Thomas Lukasiewicz, Xiaolin Hu, Jianfei Cai, and Zhenghua Xu. 2021. Rsg: A simple but effective module for learning imbalanced datasets. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 3784–3793.
- [49] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *International conference on machine learning*. PMLR, 6861–6871.
- [50] Lirong Wu, Haitao Lin, Zhangyang Gao, Cheng Tan, Stan Li, et al. 2021. Graphmixup: Improving class-imbalanced node classification on graphs by self-supervised context prediction. *arXiv preprint arXiv:2106.11133* (2021).
- [51] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
- [52] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.
- [53] Ziyu Xu, Chen Dan, Justin Khim, and Pradeep Ravikumar. 2020. Class-weighted classification: Trade-offs and robust approaches. In *International conference on machine learning*. PMLR, 10544–10554.
- [54] Divin Yan, Gengchen Wei, Chen Yang, Shengzhong Zhang, and Zengfeng Huang. 2023. Rethinking Semi-Supervised Imbalanced Node Classification from Bias-Variance Decomposition. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=0gvtohxvMY>
- [55] Yu Yang, Hao Kang, and Baharan Mirzasoleiman. 2023. Towards sustainable learning: Coresets for data-efficient deep learning. In *International Conference on Machine Learning*. PMLR, 39314–39330.
- [56] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.
- [57] Sukwon Yun, Kibum Kim, Kanghoon Yoon, and Chanyoung Park. 2022. Lte4g: long-tail experts for graph neural networks. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 2434–2443.
- [58] Hongyi Zhang, Moustapha Cissé, Yann N. Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*.
- [59] Yifan Zhang, Bingyi Kang, Bryan Hooi, Shuicheng Yan, and Jiashi Feng. 2023. Deep long-tailed learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 9 (2023), 10795–10816.
- [60] Yongshun Zhang, Xiu-Shen Wei, Boyan Zhou, and Jianxin Wu. 2021. Bag of tricks for long-tailed visual recognition with deep convolutional neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 3447–3455.
- [61] Tianxiang Zhao, Xiang Zhang, and Suhang Wang. 2021. Graphsmote: Imbalanced node classification on graphs with graph neural networks. In *Proceedings of the 14th ACM international conference on web search and data mining*. 833–841.
- [62] Mengting Zhou and Zhiguo Gong. 2023. GraphSR: A Data Augmentation Algorithm for Imbalanced Node Classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4954–4962.
- [63] Hao Zhu and Piotr Koniusz. 2020. Simple spectral graph convolution. In *International conference on learning representations*.
- [64] Wei Zhuo, Zemin Liu, Bryan Hooi, Bingsheng He, Guang Tan, Rizal Fathony, and Jia Chen. 2024. Partitioning Message Passing for Graph Fraud Detection. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=tEgrUrUuWA>

## A Proofs for Theoretical Analysis

In this section, we present theoretical proofs for the equations used in the manuscript.

### A.1 Gradient Derivation of the Cross-Entropy Loss Function

To derive Eq. 6, we need to find the derivative of the cross-entropy loss function  $L[\cdot]$ , as defined in Eq. 7, with respect to the model parameters  $\theta$ . More generally, when dealing with a subset of  $p$  nodes represented by  $\tilde{A} \in \mathbb{R}^{p \times n}$  and their associated labels  $Y \in [0, 1]^{p \times c}$ , the computation of the loss function can be broken down into several steps. At the first step, we calculate the output logits of the GNN encoder  $S \in \mathbb{R}^{p \times c}$ :

$$S = \tilde{A}X\theta \quad (21)$$

Next, we employ element-wise exponential operation to  $S$  and obtain  $E \in \mathbb{R}^{p \times c}$ :

$$E = \exp(S) \quad (22)$$

Then, we normalize  $E$  along each row and get  $H \in \mathbb{R}^{p \times c}$ , which can be seen as the softmax of the output logits  $S$ :

$$H = E \oslash J \quad (23)$$

where  $\oslash$  denotes the Hadamard division and  $J \in \mathbb{R}^{c \times c}$  represents an all-one matrix. After that, we apply the element-wise logarithm

to  $H$  and get  $Z \in \mathbb{R}^{p \times c}$

$$Z = \log(H) \quad (24)$$

Finally, the cross-entropy loss  $L$  is calculated as follows:

$$L = -\langle Y, Z \rangle_F \quad (25)$$

where  $\langle \cdot, \cdot \rangle_F$  denotes the Frobenius inner product. For simplicity, the symbol  $:$  can be used to represent the Frobenius inner product  $\langle \cdot, \cdot \rangle_F$ . Henceforth, these two symbols will be used interchangeably to indicate the Frobenius inner product.

Since directly computing the derivatives  $\nabla$  is intractable and unrealistic, we choose to work in the differential form<sup>1</sup>. In particular, starting from the cross-entropy loss (Eq. 25), we have:

$$dL = -Y : dZ \quad (26)$$

Next,

$$dZ = dH \oslash H \quad (27)$$

Then,

$$\begin{aligned} dH &= dE \oslash EJ \oslash EJ \oslash EJ - d(EJ) \oslash E \oslash EJ \oslash EJ \\ &= dE \oslash EJ - (dE)J \oslash H \oslash EJ \end{aligned} \quad (28)$$

where  $\oslash$  represents the Hadamard product. After that,

$$dE = E \oslash dS \quad (29)$$

Finally,

$$dS = \tilde{A}X d\theta \quad (30)$$

Putting Eq. 26 to Eq. 30 together, we have the following:

$$\begin{aligned} dL &= -Y : dZ \\ &= -Y : dH \oslash H \\ &= -Y : (dE \oslash EJ - (dE)J \oslash H \oslash EJ) \oslash H \\ &= -Y : ((E \oslash dS) \oslash EJ - (E \oslash dS)J \oslash H \oslash EJ) \oslash H \\ &= -Y : (E \oslash dS) \oslash EJ \oslash H - (E \oslash dS)J \oslash EJ \\ &= -Y : dS - (E \oslash dS)J \oslash EJ \\ &= -Y : dS + Y : (E \oslash dS)J \oslash EJ \\ &= -Y : dS + (Y \oslash EJ)J \oslash E : (dS) \\ &= ((Y \oslash EJ)J \oslash E - Y) : (dS) \\ &= ((Y \oslash EJ)J \oslash E - Y) : \tilde{A}X d\theta \\ &= (\tilde{A}X)^T ((Y \oslash EJ)J \oslash E - Y) : d\theta \end{aligned} \quad (31)$$

In this way, we can get the gradient  $\nabla L$  of the loss function in terms of the denominator layout:

$$\begin{aligned} \nabla_{\theta} L &= \frac{dL}{d\theta} \\ &= (\tilde{A}X)^T ((Y \oslash EJ)J \oslash E - Y) \end{aligned} \quad (32)$$

Given that  $Y$  is the label matrix with each row as a one-hot vector, we can simplify Eq.32 and get:

$$\nabla_{\theta} L = (\tilde{A}X)^T (H - Y) \quad (33)$$

Hence, the proof is complete.

<sup>1</sup>We draw the inspiration from this blog <https://math.stackexchange.com/a/3850121/1237687>

## B Algorithm

**Pseudo-Code** Algorithm 1 describes our approach. First, in lines 1-8, we derive the aggregation matrix and apply the PAM algorithm to each class's context embedding space to create a balanced meta-set. In lines 10-19, we generate synthetic nodes using a MixUp-like method. Subsequently, in lines 20-21, we assign pseudo-labels to the unlabeled nodes based on the highest predicted probabilities. In lines 22-24, we compute the importance scores of the nodes and filter out the important ones according to these scores. Lastly, in lines 25-29, we apply the cross-entropy loss to the filtered nodes and update the GNN model accordingly.

**Complexity analysis** Compared to the backbone GNN model, our algorithm adds additional computations in three main areas: meta-set construction (lines 1-8), synthetic node generation (lines 10-19), and node importance computation (lines 22-24). We will examine each part individually and elaborate on their impact on computational overhead. Let  $\mathcal{V}_{tr}$  represent the set of nodes within the training set  $D_{tr}$ , and  $\mathcal{V}_{ul}$  denote the set of unlabeled nodes.

- **Meta-set construction.** The adjacency matrix  $A$  is sparse with  $|\mathcal{E}|$  non-zero elements, the degree matrix  $D$  is diagonal, and the feature matrix  $X$  is dense. Thus, computing the context embedding  $\tilde{A}X$  involves sparse-dense matrix multiplications, with a complexity of  $O(K|\mathcal{E}|d)$  where  $K$  is the number of steps and  $d$  is the feature dimension. Running the PAM algorithm on a class of size  $|\mathcal{V}^k|$  for  $\tau$  meta-samples incurs a complexity of  $O(\tau^2|\mathcal{V}^k|^2)$ . Therefore, the overall complexity of the meta-set construction procedure is  $O(K|\mathcal{E}|d + \sum_{c \in C} \tau|\mathcal{V}_c^L|^2)$ .
- **Synthetic node generation.** The number of synthetic nodes generated per class is  $O(|\mathcal{V}_{tr}|)$ . The process begins with sampling pairs of nodes, which incurs a complexity of  $O(|\mathcal{V}_{tr}|^2)$ . For each sampled pair, generating synthetic node features and labels involves simple arithmetic operations with a complexity of  $O(|\mathcal{V}_{tr}|d)$ . Furthermore, generating edges for these synthetic nodes to maintain the original degree distribution of the graph has a complexity of  $O(|\mathcal{V}_{tr}| \cdot \bar{d})$ , where  $\bar{d}$  represents the average degree of the graph. Therefore, the overall complexity of producing synthetic nodes is  $O(|\mathcal{V}_{tr}|^2 + |\mathcal{V}_{tr}|d + |\mathcal{V}_{tr}| \cdot \bar{d})$ .
- **Node importance computation.** Given that the amount of synthetic nodes is  $O(|\mathcal{V}_{tr}|)$  and these synthetic edges maintain the original degree distribution, the augmented graph with synthetic nodes consists of  $O(|\mathcal{E}|)$  edges. Moreover, generating pseudo-labels for unlabeled nodes requires a complexity of  $O(|\mathcal{V}_{ul}|c)$ . Calculating node importance scores requires computing the context embedding for the augmented graph, which incurs a complexity of  $O(K|\mathcal{E}|d)$ , and performing dense matrix multiplications and element-wise operations, which incurs a complexity of  $O(|\mathcal{V}|d \cdot c\tau + |\mathcal{V}|c^2\tau)$ . Consequently, the complexity to determine node importance is  $O(K|\mathcal{E}|d + |\mathcal{V}_{ul}|c + |\mathcal{V}|d \cdot c\tau + |\mathcal{V}|c^2\tau)$ .

Given that  $|\mathcal{E}| \gg |\mathcal{V}_{tr}|$  and  $c, \tau$ , and  $d$  are typically small constants, in a class-imbalanced node classification scenario, the added overhead for  $T$  training epochs is simplified to  $O(T \cdot K|\mathcal{E}|d)$ , which scales linearly with the number of graph edges. Therefore, the computational overhead is comparable to that of the primary GNN model, resulting in a lightweight additional computation.

**Algorithm 1** Pseudo-code of the framework

---

**Require:** Input graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, A, X)$ , class set  $\mathcal{C} = \{1, \dots, c\}$ , training set  $\mathcal{D}_{tr} = \{(v, y_v)\}$ , set of unlabeled nodes  $\mathcal{D}_{ul}$ , GNN model  $g(\cdot, \theta)$ , depth of the aggregation matrix  $K$ , teleport probability when calculating the aggregation matrix  $\alpha$ , meta-set size per class  $\tau$ , scale for loss on unlabeled nodes  $\beta$ , scale for loss on synthetic nodes  $\gamma$ , learning rate  $\kappa$ , vector with all ones  $J$ , degree matrix  $D$

**Ensure:** Trained GNN parameters  $\theta$

- 1: Calculate the aggregation matrix  $\tilde{A} = \frac{1}{K} \sum_{k=1}^K ((1 - \alpha)(D^{-\frac{1}{2}} A D^{-\frac{1}{2}})^k + \alpha I)$
- 2: Initialize the set of meta-samples  $\mathcal{M} \leftarrow \emptyset$
- 3: **for**  $c \in \mathcal{C}$  **do**
- 4:   Apply PAM algorithm to find meta-samples in the class  $\tilde{\mathcal{M}}_k \leftarrow \text{PAM}(\tilde{A}X, \mathcal{V}^k, \tau)$
- 5:    $\mathcal{M} \leftarrow \mathcal{M} \cup \tilde{\mathcal{M}}^k$
- 6: **end for**
- 7: Construct the meta-set  $\mathcal{D}_{meta}$  from  $\mathcal{M}$ .
- 8: Remove meta-samples from the training set  $\mathcal{D}_l = \mathcal{D}_{tr} \setminus \mathcal{D}_{meta}$
- 9: **while** not converged **do**
- 10:   Sample a set of node pairs  $\mathcal{S} = \{(v_s, y_s), (v_t, y_t)\}$  from the training set  $\mathcal{D}_{tr}$  based on Eq.13
- 11:   Initialize the set of synthetic nodes  $\mathcal{D}_{syn} \leftarrow \emptyset$
- 12:   **for**  $\langle (v_s, y_s), (v_t, y_t) \rangle \in \mathcal{S}$  **do**
- 13:     Sample the mixing ratio  $\lambda \sim \text{Beta}(2, 2)$
- 14:     Generate synthetic node feature  $x_{syn} = \lambda \cdot x_s + (1 - \lambda) \cdot x_t$
- 15:     Construct distribution for neighbor sampling  $A'_{syn} = \lambda \cdot A_s + (1 - \lambda) \cdot A_t$
- 16:     Generate synthetic node label  $y_{syn} = \lambda \cdot y_s + (1 - \lambda) \cdot y_t$
- 17:     Generate synthetic edges by sampling neighbors from the calculated distribution  $A_{syn} = \text{Sample}(A'_{syn})$
- 18:      $\mathcal{D}_{syn} \leftarrow \mathcal{D}_{syn} \cup \{(v_{syn}, y_{syn})\}$
- 19:   **end for**
- 20:   Obtain predictions from the GNN model  $H \leftarrow g(\mathcal{G}, \theta)$
- 21:   Generate pseudo-labels for unlabeled nodes  $\mathcal{D}_{ul}$  according to Eq.11
- 22:   Construct the set of valuable labeled nodes  $\tilde{\mathcal{D}}_l = \{(v, y_v) \in \mathcal{D}_l \mid \eta_v > 0\}$
- 23:   Construct the set of valuable unlabeled nodes  $\tilde{\mathcal{D}}_{ul} = \{(v, \hat{y}_v) \mid v \in \mathcal{D}_{ul} \text{ and } \eta_v > 0\}$
- 24:   Construct the set of valuable synthetic nodes  $\tilde{\mathcal{D}}_{syn} = \{(v_{syn}, y_{syn}) \in \mathcal{D}_{syn} \mid \eta_{v_{syn}} > 0\}$
- 25:   Compute loss for labeled nodes  $\mathcal{L}_l \leftarrow \sum_{(v, y_v) \in \tilde{\mathcal{D}}_l} \text{Cross-Entropy}(y_v, h_v)$
- 26:   Compute loss for unlabeled nodes  $\mathcal{L}_{ul} \leftarrow \sum_{(v, \hat{y}_v) \in \tilde{\mathcal{D}}_{ul}} \text{Cross-Entropy}(\hat{y}_v, h_u)$
- 27:   Compute loss for synthetic nodes  $\mathcal{L}_{syn} \leftarrow \sum_{(v_{syn}, y_{syn}) \in \tilde{\mathcal{D}}_{syn}} \text{Cross-Entropy}(y_{syn}, h_{v_{syn}})$
- 28:   Compute the final loss  $\mathcal{L} \leftarrow \mathcal{L}_l + \beta \cdot \mathcal{L}_{ul} + \gamma \cdot \mathcal{L}_{syn}$
- 29:   Update the GNN model  $\theta \leftarrow \theta - \kappa \cdot \nabla_{\theta} \mathcal{L}$
- 30: **end while**
- 31: **return**  $\theta$

---